Ex.No:	IMPLEMENTATION OF LOGISTIC REGRESSION ALGORITHM FOR CLASSIFICATION

AIM:

To predict whether a loan application will be approved or rejected using a Logistic Regression classifier, and to evaluate the model performance using classification metrics such as Accuracy, Precision, Recall, and F1-score, along with visualization of the sigmoid function and training cost reduction graph.

PROCEDURE:

Description of the dataset:

The Loan Approval dataset contains financial and personal information of applicants used to determine whether a loan should be approved or rejected. Each record represents one loan applicant and includes attributes such as number of dependents, education level, employment status, annual income, loan amount, loan term, CIBIL score, and asset values.

These attributes help analyze the financial stability of an applicant and predict the loan approval status using machine learning classification techniques.

Total No. of Attributes: 13
Total No. of Records: 4269

Class Label (Target Attribute): loan_status

Class Distribution:
0 = 1611 (Loan Rejected)
1 = 2658 (Loan Approved)

Dataset Link:

Loan Approval Dataset
https://www.kaggle.com/datasets/architsharma01/loan-approval-prediction-dataset

Step 1: Import all the required libraries including NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn for data preprocessing, visualization, and model building.

Step 2: Load the dataset loan_approval_dataset_lab.csv into a pandas DataFrame and display the dataset shape and column names.

Step 3: Identify the target column loan_status and display the class distribution of approved and rejected loan applications.

Step 4: Remove unnecessary columns such as loan_id, which do not contribute to prediction.

Step 5: Convert categorical variables such as education and self_employed into numerical values using Label Encoding.

Step 6: Separate the dataset into feature variables (X) and target variable (y) by dropping the loan status column from the input features.

```
Confusion Matrix:
  True Negatives:  710
  False Positives: 9
  False Negatives: 120
  True Positives:  9

Precision: 50.00%
Recall: 6.98%
F1-Score: 12.24%

============================================================
SAMPLE PREDICTIONS
============================================================

Sample   Probability     Prediction   Actual
-------------------------------------------
1        0.1958          0            0
2        0.2159          0            0
3        0.2027          0            1
4        0.5138          1            0
5        0.2091          0            0
6        0.1873          0            0
7        0.2493          0            0
8        0.0905          0            0
9        0.1399          0            0
10       0.1811          0            0
```

```
============================================================
FEATURE IMPORTANCE (Weights)
============================================================
        Feature    Weight
            age  0.553967
          sysBP  0.291769
        Feature    Weight
            age  0.553967
          sysBP  0.291769
            age  0.553967
          sysBP  0.291769
          sysBP  0.291769
     cigsPerDay  0.274075
           male  0.219591
        glucose  0.161297
        totChol  0.099057
   prevalentHyp  0.090062
prevalentStroke  0.081996
          BPMeds  0.058418
       diabetes  0.024645
      heartRate -0.014018
          diaBP  0.008773
  currentSmoker  0.007127
            BMI -0.005497
      education -0.003854

✓ Cost history plot saved as 'cost_history.png'
```

Step 7: Check the dataset for missing values in each column.

Step 8: Handle missing values using median imputation for numerical columns.

Step 9: Apply feature scaling using StandardScaler to standardize the input features and improve model performance.

Step 10: Split the dataset into training and testing sets using train_test_split to evaluate the model on unseen data.

Step 11: Define the sigmoid function, which maps input values into probabilities between 0 and 1.

Step 12: Plot the sigmoid wave (S-curve) to visualize the activation function used in Logistic Regression.

Step 13: Train the Logistic Regression model using the training dataset.

Step 14: Use the trained model to predict loan approval status for the test dataset.

Step 15: Compare the predicted class labels with the actual test labels.

Step 16: Compute the confusion matrix values such as True Positives, True Negatives, False Positives, and False Negatives.

Step 17: Calculate evaluation metrics including Accuracy, Precision, Recall, and F1-Score to measure model performance.

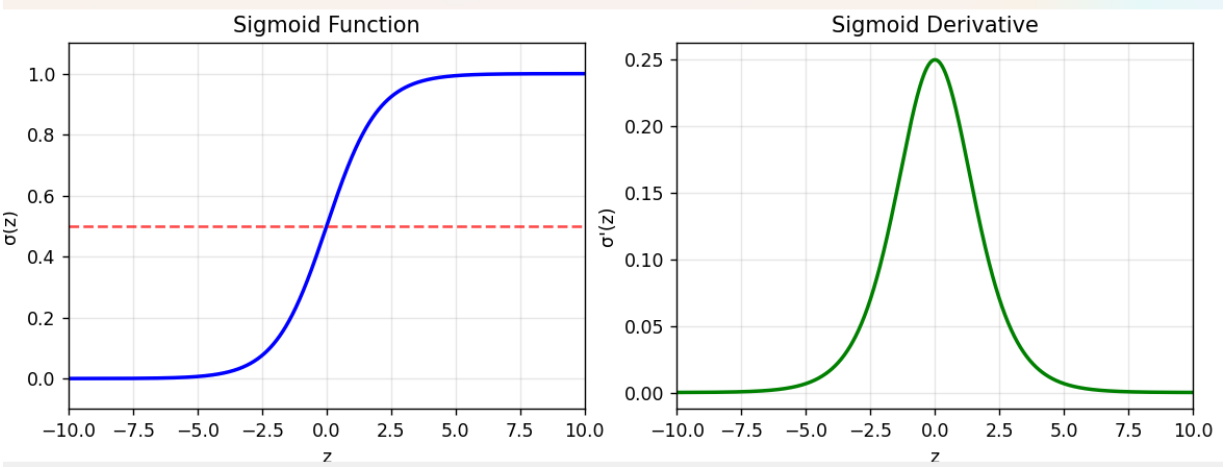
Step 18: Plot the confusion matrix heatmap to visualize the prediction results.

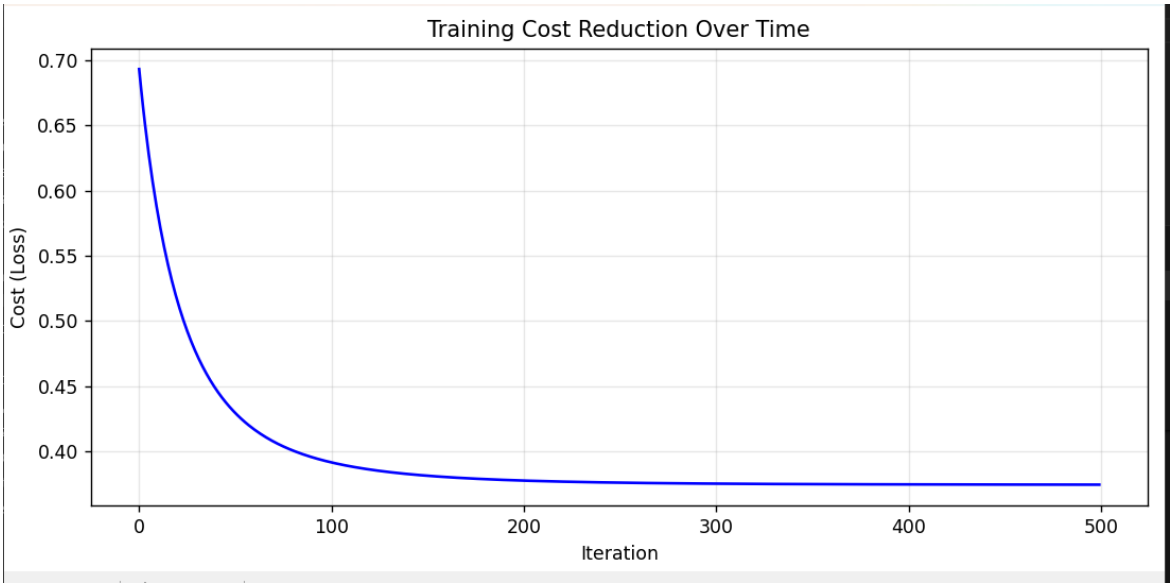
Step 19: Generate the ROC curve to evaluate the classifier performance.

Step 20: Plot the training cost reduction graph to visualize how the loss decreases during model training.

PROGRAM:

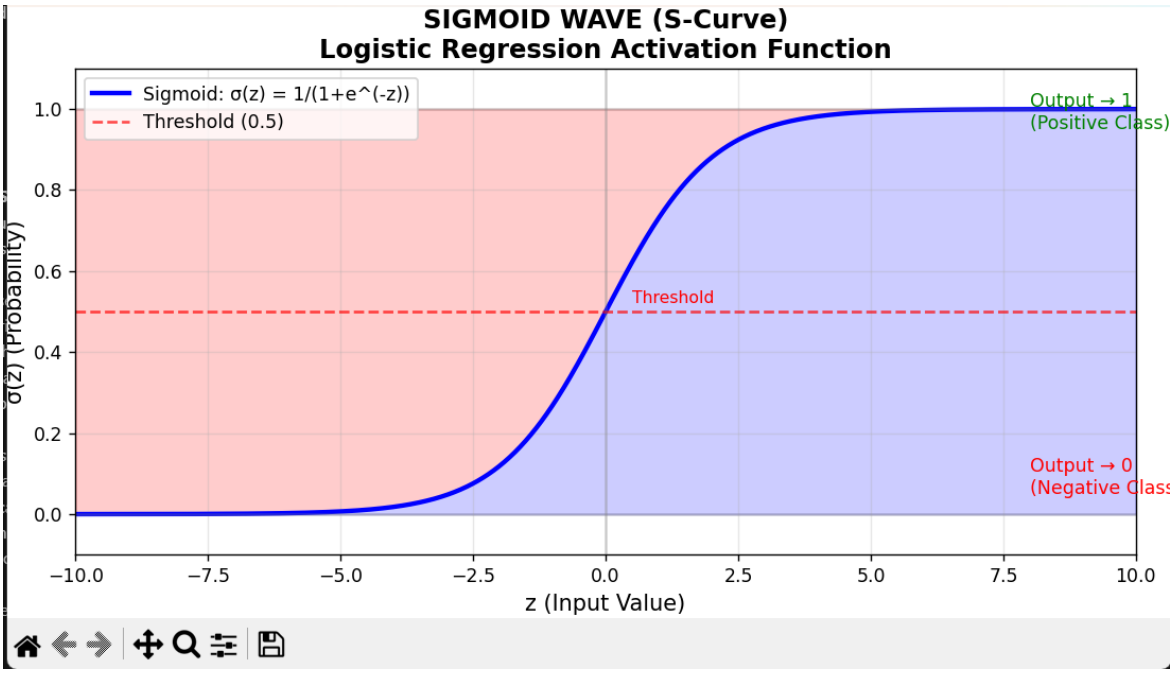
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression as SklearnLogistic
from sklearn.metrics import (
```


Sigmoid Function
σ(z)
z
Sigmoid Derivative
σ'(z)
z


Training Cost Reduction Over Time
Cost (Loss)
Iteration

```
cat_cols = [
    'gender',
    'married',
    'education',
    'self_employed',
    'property_area',
]

le = LabelEncoder()
for col in cat_cols:
    df[col] = le.fit_transform(df[col].astype(str))

df['loan_status'] = df['loan_status'].map({'Approved': 1, 'Rejected': 0})

X = df.drop('loan_status', axis=1)
y = df['loan_status']

print('\nMissing values before:')
print(X.isnull().sum())

for col in X.columns:
    if X[col].dtype in ['float64', 'int64']:
        X[col].fillna(X[col].median(), inplace=True)
    else:
        X[col].fillna(X[col].mode()[0],  inplace=True)

print('\nMissing values after:')
print(X.isnull().sum())

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_valid, y_train, y_valid = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)

model = SklearnLogistic(solver='liblinear')
model.fit(X_train, y_train)

new_applicants = pd.DataFrame([
    {
        'gender': 'Male',
        'married': 'Yes',
        'dependents': 0,
        'education': 'Graduate',
        'self_employed': 'No',
        'applicantincome': 5000,
        'coapplicantincome':  0,
        'loanamount': 150,
        'loan_amount_term':  360,
        'credit_history': 1.0,
        'property_area': 'Urban'
```

```
    {
        'gender': 'Female',
        'married': 'No',
        'dependents': 2,
        'education': 'Not Graduate',
        'self_employed': 'Yes',
        'applicantincome': 2500,
        'coapplicantincome': 1500,
        'loanamount': 120,
        'loan_amount_term': 120,
        'credit_history': 0.0,
        'property_area': 'Rural',
    },
])

for col in cat_cols:
    new_applicants[col] = le.transform(new_applicants[col].astype(str))

new_applicants_scaled = scaler.transform(new_applicants)

predictions = model.predict(new_applicants_scaled)
probabilities = model.predict_proba(new_applicants_scaled)[:, 1]

print('\nNew applicant predictions:')
for i in range(len(new_applicants)):
    print(f"Applicant {i+1}: probability={probabilities[i]:.3f}, predicted={predictions[i]}")

y_pred_valid = model.predict(X_valid)
cm = confusion_matrix(y_valid, y_pred_valid)
print('\nConfusion matrix:\n', cm)

accuracy = accuracy_score(y_valid, y_pred_valid)
precision = precision_score(y_valid, y_pred_valid)
recall = recall_score(y_valid, y_pred_valid)
f1 = f1_score(y_valid, y_pred_valid)
print(f"\nAccuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1 score: {f1:.4f}")

plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Rejected','Approved'],
            yticklabels=['Rejected','Approved'])
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.title('Confusion Matrix')
plt.tight_layout()
plt.savefig('loan_confusion_matrix.png', dpi=150)
plt.show()

y_score = model.predict_proba(X_valid)[:, 1]
fpr, tpr, _ = roc_curve(y_valid, y_score)
```


SIGMOID WAVE (S-Curve)
Logistic Regression Activation Function
Sigmoid: σ(z) = 1/(1+e^(-z))
Threshold (0.5)
Output → 1
(Positive Class)
Threshold
Output → 0
(Negative Class
σ(z) (Probability)
z (Input Value)
1.0
0.8
0.6
0.4
0.2
0.0
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
7.5
10.0

```
roc_auc = auc(fpr, tpr)
plt.figure(figsize=(6, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2,
        label=f'ROC curve (area = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=1, linestyle='--')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic')
plt.legend(loc='lower right')
plt.tight_layout()
plt.savefig('loan_roc_curve.png', dpi=150)
plt.show()

print('\nDone.')
```

RESULT:

Thus, the Logistic Regression algorithm was successfully implemented to predict the loan approval status using the given dataset.